

Guardian FS

Detección y Mitigación de Ransomware
en Linux mediante FUSE + ZFS + ML

Universidad Tecnológica Nacional — Facultad Regional Santa Fe
Proyecto Final — Primer Cuatrimestre 2026

Autores: Gómez Enrico, Ivo · Kuchen, Francisco
Director: Dr. Pablo Pessolani | Codirector: Ing. David Harispe

Versión v0.1.0 (unreleased) — Licencia MIT

Documento generado: 25 de June de 2026

Índice

1. Resumen Ejecutivo
 2. El Problema: Ransomware en Linux
 3. Solución Propuesta: Guardian FS
 4. Arquitectura del Sistema
 5. Módulos y Componentes
 6. Flujo de Detección y Mitigación
 7. Modelo de Amenazas
 8. Métricas y Objetivos
 9. Stack Tecnológico
 10. Estado del Proyecto
 11. Trabajo Pendiente
- Apéndice: Árbol del Proyecto

1. Resumen Ejecutivo

Guardian FS es un sistema de detección y mitigación de ransomware cifrador (cryptoware) desarrollado como proyecto final de la carrera de Ingeniería en Sistemas de Información en la UTN Facultad Regional Santa Fe, durante el primer cuatrimestre de 2026.

El sistema opera como un proxy transparente de sistema de archivos utilizando FUSE (Filesystem in Userspace), interceptando todas las operaciones de escritura, lectura, renombrado y eliminación antes de que lleguen al disco real. Esta intercepción permite analizar en tiempo real el comportamiento de cada proceso mediante:

- Análisis estadístico local de entropía (Shannon, chi-cuadrado, correlación) con latencia < 1ms.
- Scoring ponderado por proceso con ventanas temporales de 5 segundos.
- Inferencia por Machine Learning asíncrona vía Unix Domain Socket (Random Forest + XGBoost + Isolation Forest + LSTM).
- Archivos señuelo (canary files) como trampas de detección inmediata.
- Snapshots ZFS inmutables para recuperación instantánea ante ataques confirmados.

El proyecto se encuentra en fase temprana de desarrollo (v0.1.0, 2 commits). El código fuente existe para todos los módulos C principales (~729 líneas) y los servidores Python de ML (~404 líneas), aunque el código C aún no compila por falta de headers, build system y algunas funciones auxiliares.

2. El Problema: Ransomware en Linux

El ransomware es una de las amenazas cibernéticas más destructivas del panorama actual. En Linux, familias como LockBit 3.0, Cl0p y RansomEXX han demostrado capacidad de cifrar sistemas de archivos completos en cuestión de minutos, causando pérdidas millonarias. Las soluciones tradicionales de antivirus basadas en firmas son insuficientes frente a variantes polimórficas y ataques dirigidos.

El desafío específico que aborda este proyecto es la detección temprana: identificar un ataque de ransomware en los primeros segundos, antes de que el cifrado masivo comprometa datos críticos. El tiempo de reacción es la variable clave: cada segundo de demora puede significar cientos de archivos cifrados irrecuperablemente.

Características distintivas del ransomware cifrador que explota el sistema:

- Alta entropía en los datos escritos (bytes cifrados $\approx$ distribución uniforme, $H \approx 7.8$ bits/byte).
- Alta tasa de operaciones de escritura en cortos intervalos de tiempo.
- Patrón de lectura-escritura: lee archivo original $\rightarrow$ cifra en memoria $\rightarrow$ escribe versión cifrada.
- Cambio masivo de extensiones (.doc $\rightarrow$.locked, .pdf $\rightarrow$.encrypted).
- Procesamiento en orden alfabético o por fecha de modificación.

3. 3

Guardian FS combina tres tecnologías complementarias en una arquitectura de defensa en profundidad:

3.1 FUSE — Intercepción Transparente

Un sistema de archivos implementado en espacio de usuario que actúa como proxy entre las aplicaciones y el filesystem ZFS real. Cada syscall (open, read, write, rename, unlink) es interceptada y analizada antes de propagarse al disco. La elección de FUSE sobre un módulo de kernel prioriza seguridad (un fallo no paniquea el sistema) y velocidad de desarrollo.

3.2 Análisis Estadístico + Machine Learning

Dos capas de detección: una capa rápida en C (scoring local, latencia < 1ms) que evalúa entropía, tasas y patrones por PID; y una capa asíncrona en Python que corre un ensemble de 4 modelos ML (Random Forest, XGBoost, Isolation Forest, LSTM) sobre 14 features extraídas de cada ventana temporal. El scoring local bloquea ataques obvios inmediatamente; el ML proporciona una segunda opinión para casos borderline.

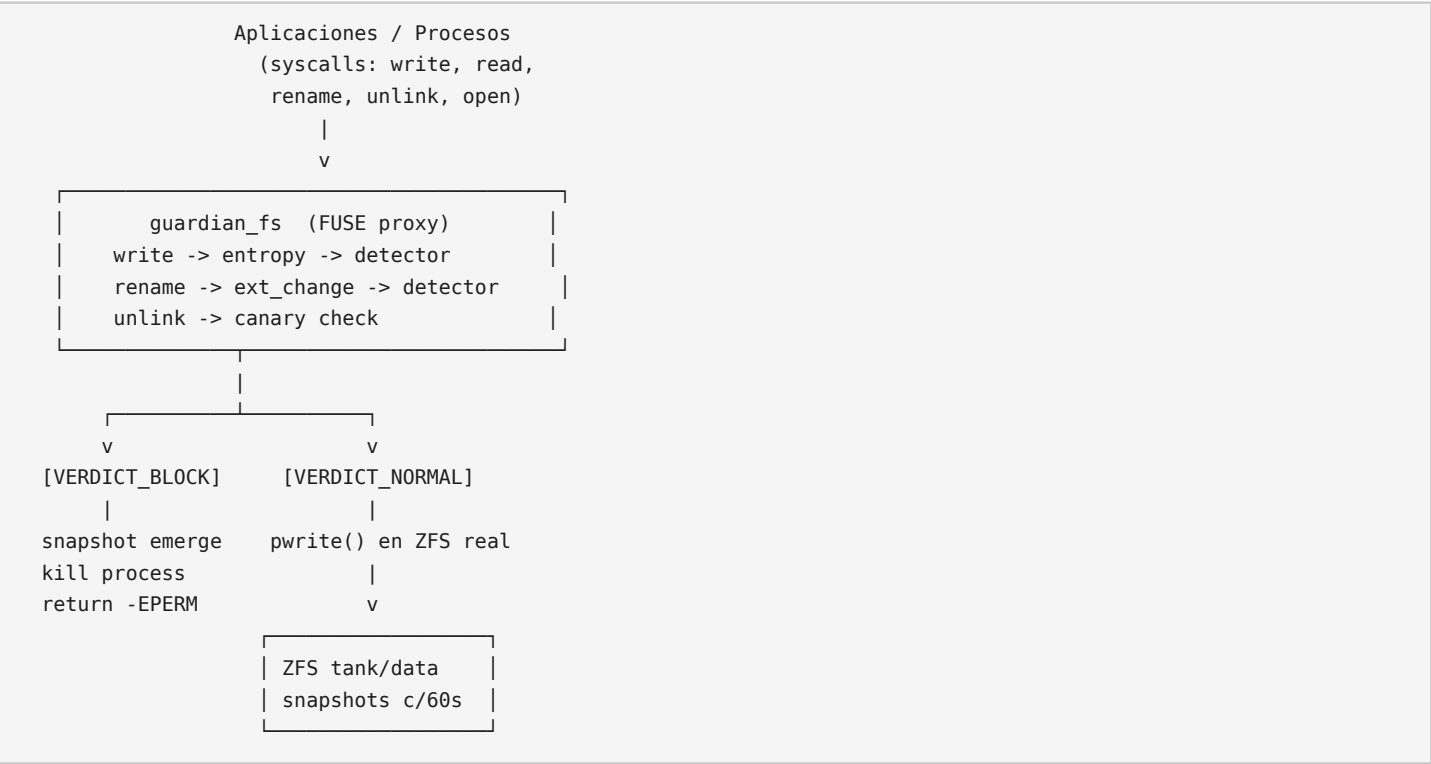
3.3 ZFS — Recuperación Inmediata

ZFS (Zettabyte File System) proporciona snapshots atómicos e inmutables mediante Copy-on-Write. Ante un ataque detectado, se toma un snapshot de emergencia y se notifica al administrador para realizar rollback al estado pre-ataque. La recuperación es O(1) en metadatos y no requiere restaurar desde backup.

4. Arquitectura del Sistema

La arquitectura sigue un patrón de capas con responsabilidades bien definidas. A continuación se detalla cada capa y su función.

4.1 Diagrama de Capas



4.2 Modelo de Hilos

Hilo	Origen	Propósito
Principal	fuse_main()	Atiende syscalls FUSE sincrónicamente
Analizador	pthread_create en main()	Procesa ring buffer, envía features a ML server
Snapshots	zfs_snapshot_schedule()	Snapshots periódicos cada N segundos (default 60s)
ML Server	ml_server.py	Múltiples hilos por conexión (1 por cliente socket)

4.3 Comunicación IPC

La comunicación entre el daemon C (`guardian_fs`) y el servidor Python de ML se realiza mediante Unix Domain Socket en `/run/guardian_ml.sock`, usando protocolo JSON-lines bidireccional. El socket tiene permisos `0o600` para seguridad local.

5. Módulos y Componentes

El proyecto consta de 8 archivos fuente principales: 6 en C y 2 en Python.

5.1 Módulos C (729 líneas)

5.1.1 `entropy.c` — Análisis Estadístico de Bytes

Implementa 4 funciones de análisis de entropía sobre buffers de bytes:

Función	Señal detectada	Complejidad	Interpretación
<code>entropy_shannon()</code>	Entropía de Shannon	$O(n)$	Cifrado ≈ 7.8 bits/byte
<code>entropy_chi_square()</code>	Uniformidad de bytes	$O(n)$	Bajo $\chi^2 \rightarrow$ uniforme $\rightarrow$ cifrado
<code>entropy_sliding_window()</code>	Cifrado parcial	$O(n)$	Detecta ransomware de primeros N bytes
<code>entropy_autocorrelation()</code>	Patrón temporal	$O(n)$	Cifrado produce baja autocorrelación

5.1.2 `detector.c` — Motor de Scoring Local

Mantiene estado por PID con ventanas de tiempo configurables (default 5s). Calcula un score ponderado combinando 6 señales con los siguientes pesos:

Señal	Peso	Descripción
Entropía media	0.35	Normalizada de [5.0, 8.0] a [0, 1]
Tasa de escrituras	0.20	Normalizada contra umbral configurable
Tasa de renombrados	0.15	Incluye detección de cambio de extensión
Test χ^2	0.20	Uniformidad de bytes acumulada en ventana
Ratio R/W	0.10	$R/W \approx 1.0$ sugiere patrón encrypt-in-place
Canary trigger	+0.50	Override directo si se accedió a canary

Veredictos basados en umbrales:

- `score < 0.50` $\rightarrow$ `VERDICT_NORMAL`: operación normal, pasa al disco.
- `score >= 0.50` $\rightarrow$ `VERDICT_SUSPICIOUS`: se registra para análisis ML asíncrono.
- `score >= 0.65` o `canary_triggered` $\rightarrow$ `VERDICT_BLOCK`: snapshot + kill + `EPERM`.

El detector es thread-safe mediante `pthread_mutex_t` por contexto.

5.1.3 `canary.c` — Archivos Señuelo

Despliega archivos con nombres estratégicamente elegidos para que ransomware que procesa en orden alfabético los encuentre temprano. Utiliza prefijos `A_` y `ZZ_` para cubrir ambos extremos, extensiones objetivo (`.docx`, `.pdf`, `.jpg`, `.xlsx`), y contenido simulado de baja entropía para no disparar falsos positivos. Hasta 20 canaries configurables.

5.1.4 `zfs_snap.c` — Interfaz con ZFS

Tres operaciones principales mediante `system()` y `popen()`:

- `zfs_snapshot_emergency()`: snapshot inmediato con timestamp al detectar ataque.
- `zfs_snapshot_schedule()`: hilo separado que toma snapshots periódicos, retiene últimos 20.
- `zfs_rollback_latest()`: rollback al snapshot más reciente con prefijo dado.

5.1.

Implementa el filesystem FUSE con 10 operaciones hookeadas. Es el punto de entrada del sistema (main). Monta en `/mnt/protected` y redirige a `/zpool/data`. Operaciones críticas de seguridad implementadas:

- `gfs_write()`: calcula entropía → detector → bloquea si necesario → sino `pwrite()`.
- `gfs_rename()`: detecta cambio de extensión y verifica contra detector.
- `gfs_unlink()`: si el path es un canary → bloqueo inmediato.
- `gfs_open()`: detecta apertura de canary y señala al detector.

5.2

5.2.1 ml_server.py — Servidor de Inferencia ML

Clasifica ventanas de eventos usando un ensemble de 4 modelos con voting ponderado:

Modelo	Peso	Tipo	Propósito
Random Forest	0.35	Supervisado	Clasificación binaria general, interpretable
XGBoost	0.30	Supervisado	Gradient boosting, maneja desbalance de clases
Isolation Forest	0.20	No supervisado	Detección de anomalías no vistas en entrenamiento
LSTM (PyTorch)	0.15	Series temporales	Patrones secuenciales de 10 ventanas por PID

Vector de 14 características extraídas por ventana. Incluye fallback rule-based cuando no hay modelos entrenados. Comunicación vía Unix Domain Socket con protocolo JSON-lines. Veredicto por umbrales: `p_attack ≥ 0.75` → "attack", `≥ 0.50` → "suspicious".

5.2.2 train_model.py — Entrenamiento Offline

Pipeline de entrenamiento completo:

- Carga CSV etiquetado con 14 features + label (0=benigno, 1=ransomware).
- Balanceo de clases con SMOTE (Synthetic Minority Over-sampling).
- Normalización con StandardScaler.
- Validación cruzada estratificada `k=10` con métrica ROC-AUC.
- Reporte de clasificación: FPR (False Positive Rate) y FNR (False Negative Rate).
- Gráfico de feature importance del Random Forest.
- Serialización: `scaler.pkl`, `rf.pkl`, `xgb.json`.

6.1

6.1 Operación Normal (sin ataque)

```
Aplicación escribe archivo normal
-> gfs_write(path, buf, size, offset)
-> entropy_shannon(buf) => H = 4.2 bits/byte
-> detector_check_write() => score = 0.18
-> VERDICT_NORMAL
-> pwrite() en ZFS real
-> Éxito
```

6.2 Detección de Ataque (ransomware cifrando)

```
Ransomware escribe bytes cifrados
-> gfs_write(path, buf, size, offset)
-> entropy_shannon(buf) => H = 7.9 bits/byte
-> detector_check_write()
    score = 0.35*f(7.9) + 0.20*f(alta_tasa) + ...
    score = 0.72 => VERDICT_BLOCK
-> zfs_snapshot_emergency("tank/data")
    "zfs_snapshot ...@guardian_emergency_20260415T143022Z"
-> mitigation_kill_process(pid) => SIGKILL
-> return -EPERM
-> Escritura NO persiste en ZFS
```

6.3 Detección por Canary

```
Ransomware renombra A_important_report.docx
-> gfs_rename(
    "A_important_report.docx",
    "A_important_report.locked")
-> canary_is_canary() => TRUE
-> zfs_snapshot_emergency()
-> mitigation_kill_process()
-> return -EPERM
```

6.4 ML Asíncrono (segunda opinión)

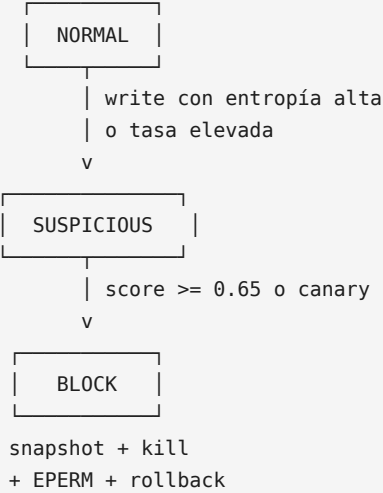
El hilo analyzer_thread (a implementar) ejecutará en segundo plano:

```
analyzer_thread (cada ~1s):
1. Procesa ring_buffer de eventos
2. Calcula 14 features por PID
3. Conecta a /run/guardian_ml.sock
4. Envía: {"features": {...}, "pid": 1234}
5. Recibe: {"p_attack": 0.89, "verdict": "attack"}
6. Si p_attack >= 0.75 => señala bloqueo al detector
```

6.5 Recuperación (Rollback)

El administrador ejecuta `zfs_rollback_latest()` que consulta el snapshot más reciente con prefijo `guardian_emergency_` y ejecuta `"zfs rollback -r"` para restaurar todo el dataset al estado inmediatamente anterior al ataque. La operación es $O(1)$ en metadatos y no requiere copia de datos.

6.6 Máquina de Estados por PID



7. Modelo de Amenazas

7.1 Activos

Activo	Valor	Mecanismo de protección
Datos del usuario	Alto	FUSE proxy + detección antes del disco
Integridad del FS	Alto	ZFS checksums sha256 + snapshots
Snapshots ZFS	Muy alto	Inmutables por CoW, solo root puede destruir
Modelos ML	Medio	Archivos en /var/lib/guardian/, permisos restrictivos

7.2 Amenazas Identificadas

ID	Amenaza	Impacto	Mitigación
T1	Ransomware evade canaries alfabéticos	Alto	Prefijos A_ y ZZ_ cubren ambos extremos
T2	Cifrado a baja velocidad	Alto	Entropía es independiente de tasa; ML + LSTM
T3	Ataque al socket Unix ML	Medio	Socket 0o600, listen(10), fallback rule-based
T4	Eliminación de canaries antes de cifrar	Alto	Canaries ocultos (.hidden_*) + gfs_unlink bloquea
T5	Falsos positivos (backup, compresión)	Medio	Score configurable, ML ensemble, modo suspicious
T6	Agotamiento pool ZFS por snapshots	Medio	Retención máxima 20 automáticos
T7	Rollback borra datos legítimos	Alto	Rollback manual, logs de eventos
T8	Bypass FUSE (acceso directo a ZFS)	Muy alto	ZFS montado fuera del namespace público
T9	Ransomware cifra solo metadatos	Medio	gfs_rename monitorea cambio de extensión
T10	ML server no disponible	Medio	Fallback a reglas fijas en C (detector.c)
T11	Ransomware con firma polimórfica	Alto	Isolation Forest detecta anomalías nuevas
T12	Ataque dirigido con conocimiento del sist	Muy alto	Defensa en profundidad (3 capas)

7.3 Defensa en Profundidad

Tres capas de defensa independientes:

- Capa 1 — Canary files: detección inmediata, latencia cero.
- Capa 2 — Scoring estadístico en C: 6 señales, latencia < 1ms, thread-safe.
- Capa 3 — ML ensemble asíncrono: 4 modelos, 14 features, segunda opinión.

El proyecto define tres variables dependientes principales como objetivos cuantitativos:

Métrica	Descripción	Objetivo
Pérdida de datos	Bytes cifrados antes del bloqueo	< 1 MB en ataque típico
RTO	Tiempo de zfs rollback (Recovery Time)	< 30 segundos
Costo almacenamiento	Overhead de snapshots ZFS	< 5% del dataset
FPR	Tasa de falsos positivos	< 1%
TPR	Tasa de detección (True Positive Rate)	> 99%
Latencia E/S	Overhead inducido por el proxy FUSE	< 2ms por operación

Familias de ransomware evaluadas en el desarrollo:

- LockBit 3.0 (Linux) — cifrado multihilo, alta velocidad.
- Cl0p (Linux variant) — evasión de detección, firma parcial.
- RansomEXX — ataque dirigido a infraestructura crítica.
- Simuladores propios — scripts de cifrado de archivos en masa (control).

Componente	Tecnología	Versión
Filesystem proxy	FUSE (libfuse3)	3.16+
Storage	OpenZFS (ZFS on Linux)	2.2+
Lenguaje principal	C (C11, GCC/Clang)	13+ / 17+
ML / Análisis	Python 3	3.11+
ML supervisado	scikit-learn RF + XGBoost	≥1.5, ≥2.0
ML no supervisado	IsolationForest (scikit-learn)	≥1.5
ML series temp.	PyTorch LSTM	≥2.0 (opcional)
Balanceo datos	imbalanced-learn (SMOTE)	≥0.12
Procesamiento	NumPy, Pandas	≥1.24, ≥2.0
Build system	CMake	3.22+
OS objetivo	Ubuntu LTS (kernel 6.8+)	24.04

10. Estado del Proyecto

El proyecto se encuentra en fase temprana (v0.1.0, 2 commits en rama develop). La arquitectura está bien definida y los módulos principales tienen implementación inicial. El código Python (ml_server.py y train_model.py) está funcionalmente completo. El código C requiere trabajo de integración para compilar.

10.1 Implementado

Módulo	Archivo	Líneas	Estado
Entropía	entropy.c	167	Esqueleto funcional (con duplicado)
Detector	detector.c	185	Esqueleto funcional
Canary	canary.c	78	Esqueleto funcional

ZFS Snap	zfs_snap.c	96	Esqueleto funcional
FUSE main	guardian_fs.c	203	Esqueleto (sin compilar)
ML Server	ml_server.py	297	Casi completo
ML Training	train_model.py	107	Casi completo
Setup script	zfs_setup.sh	48	Completamente funcional

10.2 Problemas Conocidos

1. entropy.c duplicado: las 4 funciones aparecen dos veces (líneas 1-84 y 84-167). Esto causa error de compilación por redefinición de símbolos.
2. Headers faltantes: entropy.h, detector.h, canary.h, zfs_snap.h, ring_buffer.h no existen en disco. Ningún archivo C compila.
3. clock_gettime_ns() no es POSIX estándar. Debe reemplazarse por clock_gettime() + conversión.
4. Funciones no implementadas: analyzer_thread(), mitigation_kill_process(), detector_signal_canary(), detector_check_rename(), gfs_readdir(), gfs_mkdir(), etc.
5. CMakeLists.txt no existe en disco — no hay build system.

11. Trabajo Pendiente

Brecha entre lo declarado en README.md y lo existente en disco:

- Crear headers locales (entropy.h, detector.h, canary.h, zfs_snap.h, ring_buffer.h).
- Implementar ring_buffer.c (buffer circular de 64KB para eventos de E/S).
- Implementar mitigation_kill_process() — terminación de procesos vía SIGKILL.
- Implementar analyzer_thread() — hilo asíncrono que consulta al ML server.
- Completar operaciones FUSE faltantes (readdir, mkdir, create, release, truncate).
- Crear CMakeLists.txt para build system.
- Implementar scripts: install_deps.sh, mount.sh, rollback.sh, run_tests.sh.
- Escribir suite de tests unitarios, de integración y fixtures.
- Crear archivos de configuración (guardian.conf, logging.conf).
- Configurar GitHub Actions CI/CD.
- Crear python/requirements.txt.
- Corregir duplicación en entropy.c.

UTN-FRSF-Proyecto-Final/

```
├─ README.md
├─ LICENSE
├─ CHANGELOG.md
├─ .gitignore
├─ docs/
│   └─ architecture/
│       └─ README.md
├─ scripts/
│   └─ zfs_setup.sh
└─ src/
    ├─ entropy.c      (167 líneas)
    ├─ detector.c     (185 líneas)
    ├─ canary.c       (78 líneas)
    ├─ zfs_snap.c     (96 líneas)
    ├─ guardian_fs.c (203 líneas)
    ├─ ml_server.py   (297 líneas)
    └─ train_model.py (107 líneas)
```

Total: 10 archivos, ~1,133 líneas de código

Ramas: master (estable), develop (desarrollo activo)

Commits: 2 en develop